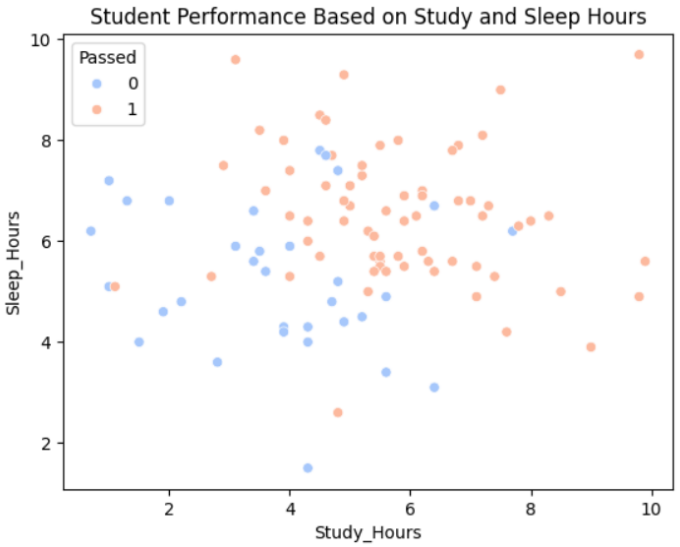
Aim: Assignment.

1.Add polynomial features or interaction terms (Study_Hours * Sleep_Hours).

CODE:

```
np.random.seed(10)
data = {
    'Study_Hours': np.random.normal(5, 2, 100).round(1),
    'Sleep_Hours': np.random.normal(6, 1.5, 100).round(1)
}
df = pd.DataFrame(data)

df['Passed'] = np.where((df['Study_Hours'] + df['Sleep_Hours']) > 10, 1, 0)

flip_indices = np.random.choice(df.index, size=10, replace=False)
df.loc[flip_indices, 'Passed'] = 1 - df.loc[flip_indices, 'Passed']

df['StudySleep_Interaction'] = df['Study_Hours'] * df['Sleep_Hours']

sns.scatterplot(data=df, x='Study_Hours', y='Sleep_Hours', hue='Passed', palette='coolwarm')
plt.title("Student Performance Based on Study and Sleep Hours")
plt.show()

X = df[['Study_Hours', 'Sleep_Hours', 'StudySleep_Interaction']]
y = df['Passed']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print('Accuracy with Interaction Term:', accuracy_score(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
print('Classification Report:\n', classification_report(y_test, y_pred))
```

OUTPUT:



```
Accuracy with Interaction Term: 0.9
Confusion Matrix:
 [[ 4  2]
 [ 0 14]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.67      0.80         6
           1       0.88      1.00      0.93        14

    accuracy                           0.90        20
   macro avg       0.94      0.83      0.87        20
weighted avg       0.91      0.90      0.89        20
```

2.Try logistic regression with only one feature and compare.

CODE:

```
X_single = df[['Study_Hours']]
y_single = df['Passed']

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_single, y_single, test_size=0.2, random_state=42)

model_single = LogisticRegression()
model_single.fit(X_train_s, y_train_s)

y_pred_s = model_single.predict(X_test_s)

print('Accuracy (Study_Hours only):', accuracy_score(y_test_s, y_pred_s))
print('Confusion Matrix:\n', confusion_matrix(y_test_s, y_pred_s))
print('Classification Report:\n', classification_report(y_test_s, y_pred_s))

print('Accuracy:', accuracy_score(y_test, y_pred))
```

OUTPUT:

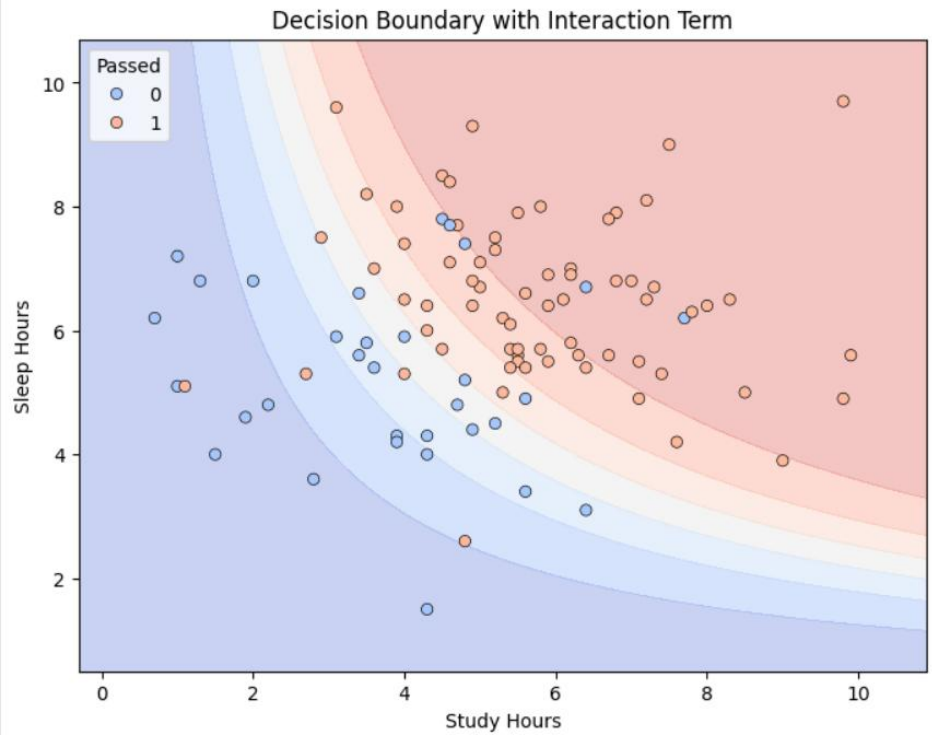
```
Accuracy (Study_Hours only): 0.7
Confusion Matrix:
 [[ 2  4]
 [ 2 12]]
Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.33      0.40         6
           1       0.75      0.86      0.80        14

    accuracy                           0.70        20
   macro avg       0.62      0.60      0.60        20
weighted avg       0.68      0.70      0.68        20

Accuracy: 0.85
```

3.Visualize the decision boundary.

CODE:

```
x_min, x_max = df['Study_Hours'].min() - 1, df['Study_Hours'].max() + 1
y_min, y_max = df['Sleep_Hours'].min() - 1, df['Sleep_Hours'].max() + 1

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

interaction = xx * yy

grid = pd.DataFrame({
    'Study_Hours': xx.ravel(),
    'Sleep_Hours': yy.ravel(),
    'StudySleep_Interaction': interaction.ravel()
})

Z = model.predict_proba(grid)[:, 1]
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8,6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
sns.scatterplot(data=df, x='Study_Hours', y='Sleep_Hours', hue='Passed', palette='coolwarm', edgecolor='k')
plt.title("Decision Boundary with Interaction Term")
plt.xlabel("Study Hours")
plt.ylabel("Sleep Hours")
plt.show()
```

OUTPUT:



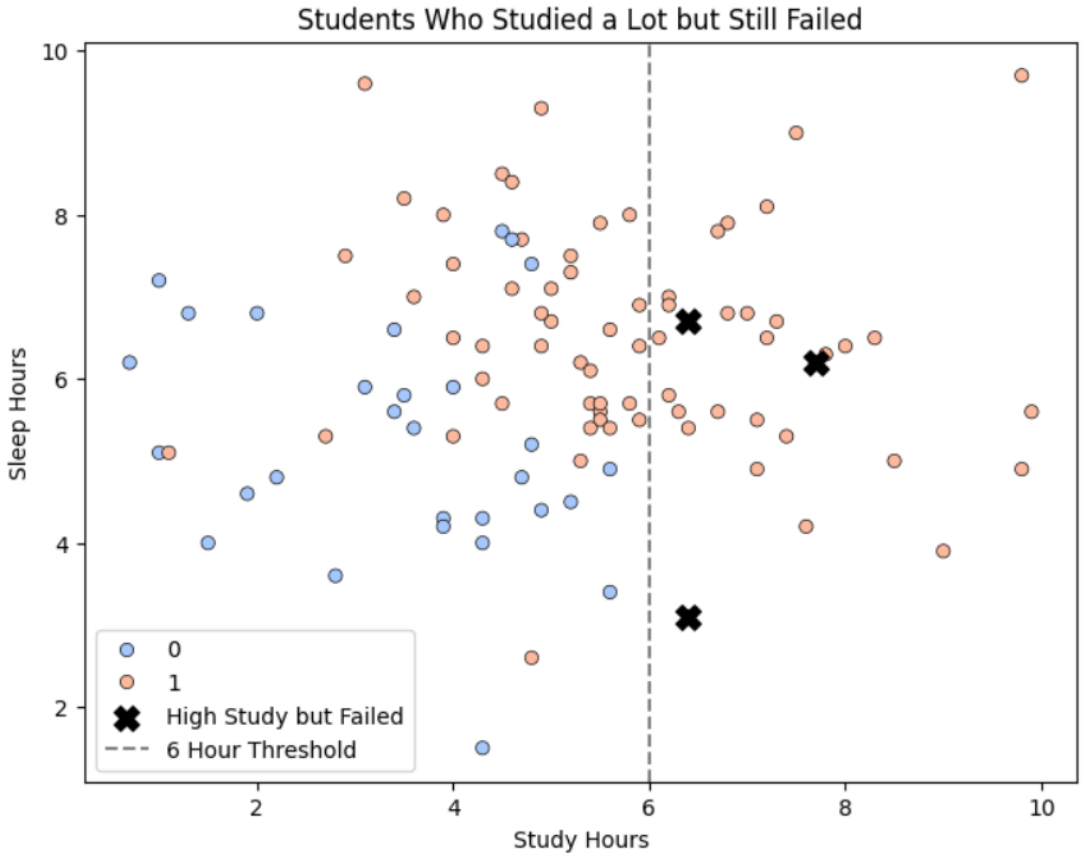
4.Can you find a student who studied a lot but still failed? What might that be?

CODE:

```
study_threshold = 6
high_study_fail = df[(df['Study_Hours'] > 6) & (df['Passed'] == 0)]
print("Students who studied a lot but failed:\n", high_study_fail)

plt.figure(figsize=(8,6))

sns.scatterplot(data=df, x='Study_Hours', y='Sleep_Hours', hue='Passed', palette='coolwarm', edgecolor='k')

plt.scatter(high_study_fail['Study_Hours'], high_study_fail['Sleep_Hours'], color='black', s=120, label='High Study but Failed', marker='X')

plt.axvline(study_threshold, color='gray', linestyle='--', label=f'{study_threshold} Hour Threshold')
plt.title("Students Who Studied a Lot but Still Failed")
plt.xlabel("Study Hours")
plt.ylabel("Sleep Hours")
plt.legend()
plt.show()
```

OUTPUT:

```
Students who studied a lot but failed:
     Study_Hours  Sleep_Hours  Passed  StudySleep_Interaction
0            7.7          6.2       0                   47.74
1            6.4          3.1       0                   19.84
53           6.4          6.7       0                   42.88
```



5.What happens if we add more noise to the labels?

CODE:

```
np.random.seed(10)
data = {
    'Study_Hours': np.random.normal(5, 2, 100).round(1),
    'Sleep_Hours': np.random.normal(6, 1.5, 100).round(1)
}
df = pd.DataFrame(data)

df['Passed'] = np.where((df['Study_Hours'] + df['Sleep_Hours']) > 10, 1, 0)

flip_indices = np.random.choice(df.index, size=30, replace=False)
df.loc[flip_indices, 'Passed'] = 1 - df.loc[flip_indices, 'Passed']

X = df[['Study_Hours', 'Sleep_Hours']]
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print('Accuracy with More Noise:', accuracy_score(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
print('Classification Report:\n', classification_report(y_test, y_pred))
```

OUTPUT:

```
Accuracy with More Noise: 0.7
Confusion Matrix:
 [[ 2  5]
 [ 1 12]]
Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.29      0.40         7
           1       0.71      0.92      0.80        13

    accuracy                           0.70        20
   macro avg       0.69      0.60      0.60        20
weighted avg       0.69      0.70      0.66        20
```

6.Try adding a new feature: "Coffee_Intake" and retrain. Does it help?

CODE:

```
np.random.seed(10)
data = {
    'Study_Hours': np.random.normal(5, 2, 100).round(1),
    'Sleep_Hours': np.random.normal(6, 1.5, 100).round(1)
}
df = pd.DataFrame(data)

df['Passed'] = np.where((df['Study_Hours'] + df['Sleep_Hours']) > 10, 1, 0)

flip_indices = np.random.choice(df.index, size=10, replace=False)
df.loc[flip_indices, 'Passed'] = 1 - df.loc[flip_indices, 'Passed']

df['Coffee_Intake'] = np.random.normal(3, 1, 100).round(1)  # Cups per day

X = df[['Study_Hours', 'Sleep_Hours', 'Coffee_Intake']]
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print('Accuracy with Coffee_Intake:', accuracy_score(y_test, y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, y_pred))
print('Classification Report:\n', classification_report(y_test, y_pred))
```

OUTPUT:

```
Accuracy with Coffee_Intake: 0.85
Confusion Matrix:
 [[ 3  3]
 [ 0 14]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.50      0.67         6
           1       0.82      1.00      0.90        14

    accuracy                           0.85        20
   macro avg       0.91      0.75      0.78        20
weighted avg       0.88      0.85      0.83        20
```